University of Southern Denmark

Mood2-D2: A deep-learning classification of texts to emotions

Machine learning project 2

AI506

Simon Holm
sihol24@student.sdu.dk

Johannes Rothe
jorot24@student.sdu.dk

Shuagib Ibrahim
shibr24@student.sdu.dk

Supervisor: Prof. Lukas Galke Poech
Date: May 20th 2026

SDU
DEPARTMENT OF MATHEMATICS AND COMPUTER SCIENCE

Contents

1 Introduction

This report explores the task of emotion classification across 6 distinct emotion labels. By implementing 2 sequence classifier models and a transformer model from scratch, the report tries to achieve high classification accuracies and in depth analysis on the results with interpretability techniques to achieve optimal models. This is done using for example, linear probing and in depth model analysis to understand the underlining representation of the models. Lastly the report will explain and discuss different possible reasons behind the findings.

1.1 Focus in this assignment

While hyper parameters play an important role in model performance, the main focus of this work is the comparison of different models architectures and interpreting the results, which is the most interesting part of this assignment. For this reason, this paper does not delve into a detailed hyper parameter analysis. However the key parameters for each model will still be reported and justified where relevant, as they directly affect the comparisons made in this work. Note also that all models have been reasonably tuned to ensure fair comparisons and stable training across experiments.

2 Data preparation / Dataloader

The `dataloader.py` is not only responsible for loading data for the models, but also for analysis of the data itself. It's very relevant for any project dealing with datasets like this to analyze the data, and learn what the data looks like from a high abstraction layer.

First of all, one should analyze the distribution of the data. For this, we assume that the dataset is well-behaved, meaning that every input has a corresponding label. This is a reasonable assumption given that the data is sourced from `dair-ai/emotion`, which is a curated dataset intended for supervised emotion classification. In otherwise cases, it would be necessary to validate label completeness and address any missing or noisy data.

With that assumption, lets now compare the splits by counter the labels of each split:

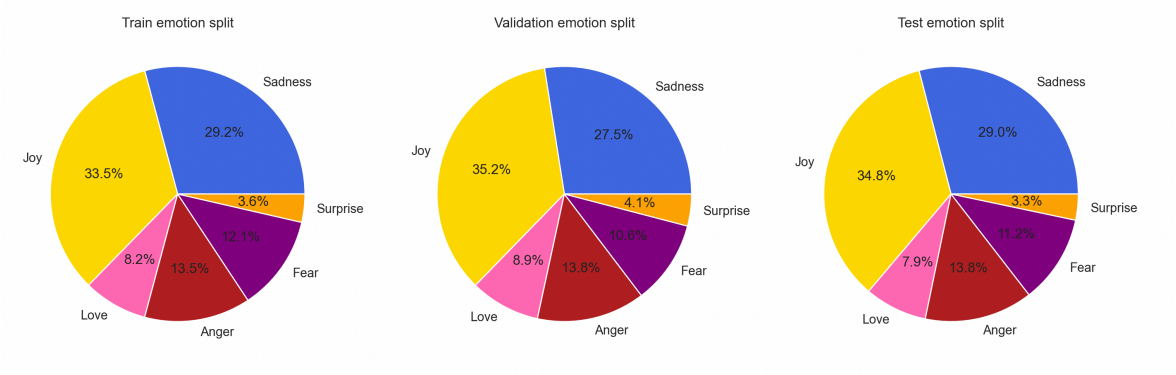

Figure 1: Distribution of labels in each split of data. The classes 'Joy' and 'Sadness' are significantly overrepresented compared to other classes e.g. 'Surprise'

h `train: 16000, validation: 2000, test: 2000`

The splits very clearly are biased towards the classes `joy` and `sadness`, this is not much of importance as long as it's addressed when reviewing model predictions.

Any model trained on this data will automatically (and without fault of the implementation) be biased towards some classes. Classes such as `surprise` are very much underrepresented in the dataset, which could impact the models ability to classify them correctly.

When choosing a model for tasks such as this, the length of the sentences (sequences) is also of relevance, as different models might struggle with, for example long sentences. Lets first extract mean, variance, std variation and range of the dataset:

$$\mu = \frac{1}{N}\sum_i x_i \qquad \sigma^2 = \frac{1}{N}\sum_i (x_i - \mu)^2 \qquad \sigma = \sqrt{\frac{1}{N}\sum_i (x_i - \mu)^2} \qquad r(x) = [\min(x), \max(x)]$$

```
1  19.154 121.21128400000002 11.009599629414325 (3, 61)                output
```

These results indicate that most sentences are relatively short, with an average length of ≈ 19 words. However, there is still noticeable variability, as sentence lengths range from very short inputs (3 words) to significantly longer ones (up to 61 words). This spread suggests that the model must be able to handle variable-length sequences effectively, while also being robust to shorter, more common inputs.

The calculations above, are taken over the whole dataset, however the distribution of sequence-lengths is also an interesting factor for each individual set.

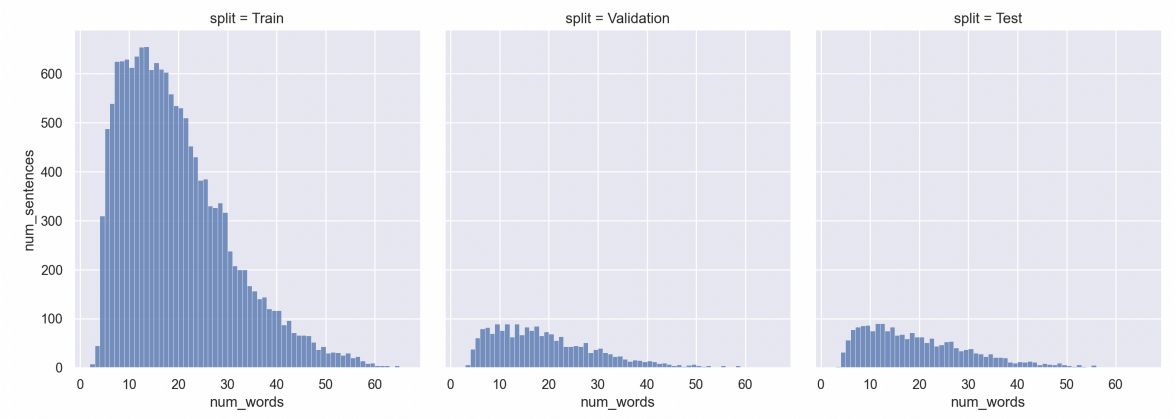

Figure 2: Distribution of sequence lengths for each split of data. Showing that the the overall shape of the sentence length distribution is the same for each split.

These 3 distributions share notable similarity. While there is much more data in the train split the overall shape of the length distribution indicate a nicely randomized split. If the data did not affirm this, then even a well-trained model would be biased towards either short or long sentences, and could then perform poorly on the sentence lengths it wasn't trained on.

2.1 Tokenization + Vocabulary

One can use various methods to tokenize the corpus, including word, character, and subword tokenization. Given the size of the dataset as well as the difficulty of assessing improvements of the task, word tokenization was chosen as a candidate. This methods does whitespace words tokenization, tokenizing sentences like `'i like cake'` to `'i', 'like', 'cake`. There could be other more optimal methods for tokenization, but given the scope of the project, this was deemed to optimal solution.

The `Vocabulary` consists of two python dictionaries: `token2idx` and `idx2token`, which acts as a mapping from token to an index and back. This way, any sentence can be encoded to integers aback to word, using the vocabulary. Take the vocabulary

```
token2idx = {'i': 2, 'am': 3, 'happy': 4}
idx2token = {2: 'i', 3: 'am', 4: 'happy'}
>>> encode([['i', 'am', 'happy']])
[[2, 3, 4]]
>>> decode(encode([['i', 'am', 'happy']]))
[['i', 'am', 'happy']]
```

3 Models

The overall objective of the models is to optimize their parameters, such as minimizing the loss associated with predicting the correct class. Since all the used models will use *Cross Entropy Loss*, the overall function can be stated as:

$$\ell(\theta) = -\sum_{i}^{n}\sum_{c}^{C} y_{i,c} \cdot \log\Big(h_\theta(x_i)_c\Big)$$

Where:

$$\begin{aligned} n &= \text{data} \\ C &= \text{classes} \\ y &= \text{target vector} \\ \theta &= \text{learnable parameters} \\ h_\theta &= \text{the model} \\ x_i &= \text{data point} \end{aligned}$$

This project implemented an RNN, LSTM and Transformer. These models are of course different in their performance and intended use cases, so by implementing all of them, useful comparisons could be drawn between, enabling for a rigorous test of theory and inherent bias what *should* perform the best, compared to what ended up *actually* performing the best.

3.1 Model 1

The very first intuition was to build a RNN model, since they are generally considered good for text analysis and would work as a stepping stone towards more complex models. RNN's work well in text analysis because of its general recurrence formula:

$$h_t = \tanh(W_{hh} \cdot h_{t-1}) + W_{xh} \cdot x_t + b$$

These type of neural networks behave well on texts because of *order-sensitive summary*, where all hidden states (tokens) are represented in the final state. This representation gives a vastly different flow of information from input → output than more ordinary neural networks:

For model 1, lets look at a RNN architecture usign pytorch.

```python
def forward(self,
input_ids, lengths):
        emb = self.embedding(input_ids)

        # ignore all packed hidden states
        packed = pack_padded_sequence(emb, lengths,
                                      batch_first=True,
                                      enforce_sorted=False)
        # apply rnn (only use last hidden state)
        _, hidden = self.rnn(packed)

        # only last layer (num_layers, B, H) → (B, H)
        hidden = hidden[-1]

        # Linear classifier  (B, H) → (B, num_classes)
        return self.fc(pooled)
```

Code snippet 1: Basic RNN architecture
embedding → packing → rnn → unsqueeze → classify

3.1.1 Hidden and Embedding dimensions:

To find the optimal model, it was necessary to find the optimal amount of hidden and embedding layers, that were found experimentally. The following figure shows the performance of a RNN with a fixed embedding dimension of 128, and hidden dimensions of 32, 64 and 128:

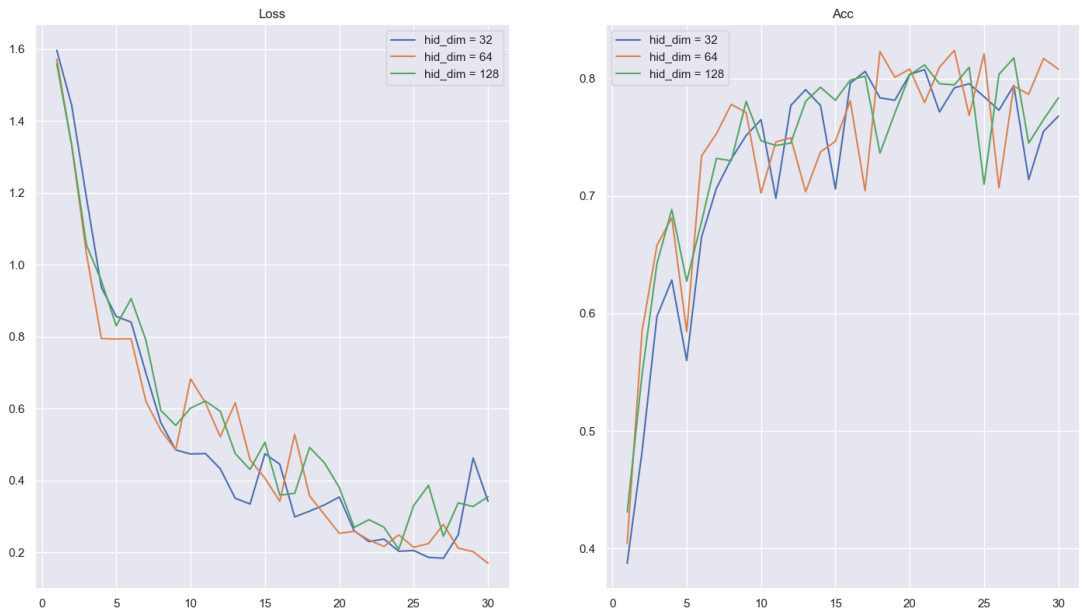

Figure 3: Testing RNN with hidden dimensions of 32, 64 and 128, with a fixed embedding dimension of 128. Showing that hidden dimension = 64 performs the best.

From the testing it is clear that having a hidden dimension of 64 performed the best, so this was chosen as the default parameter value. Then the embedding dimensions could be tested with a fixed hidden dimension of 64:

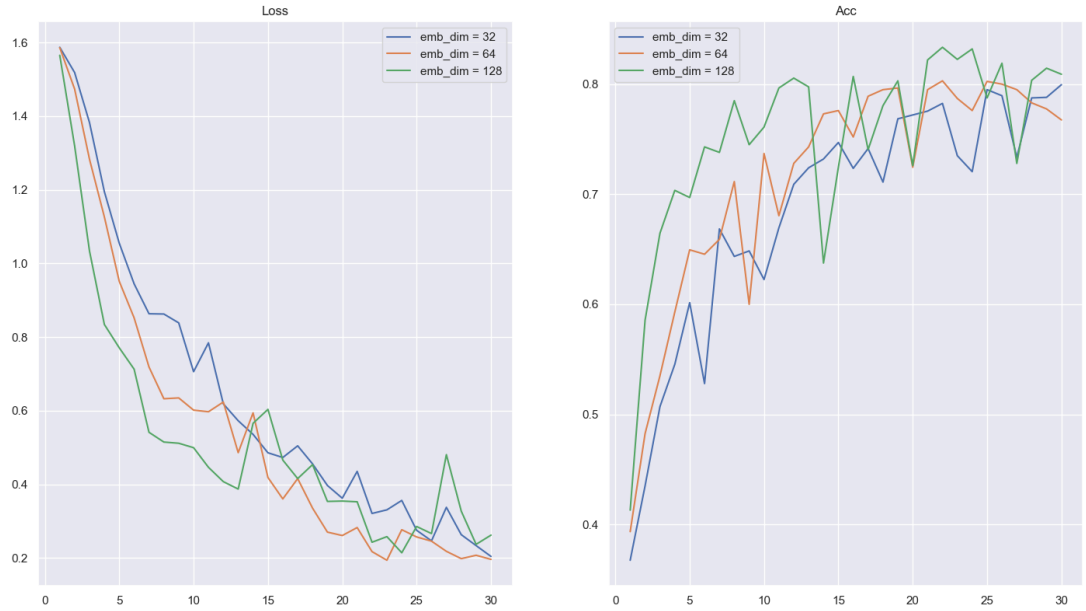

Figure 4: Testing RNN with embedding dimensions of 32, 64 and 128, with a fixed hidden dimension of 64. Showing that embedding dimension = 128 performs the best.

From the testing it can be determined that an embedding of 128 performed the best, so this was chosen as the default parameter value.

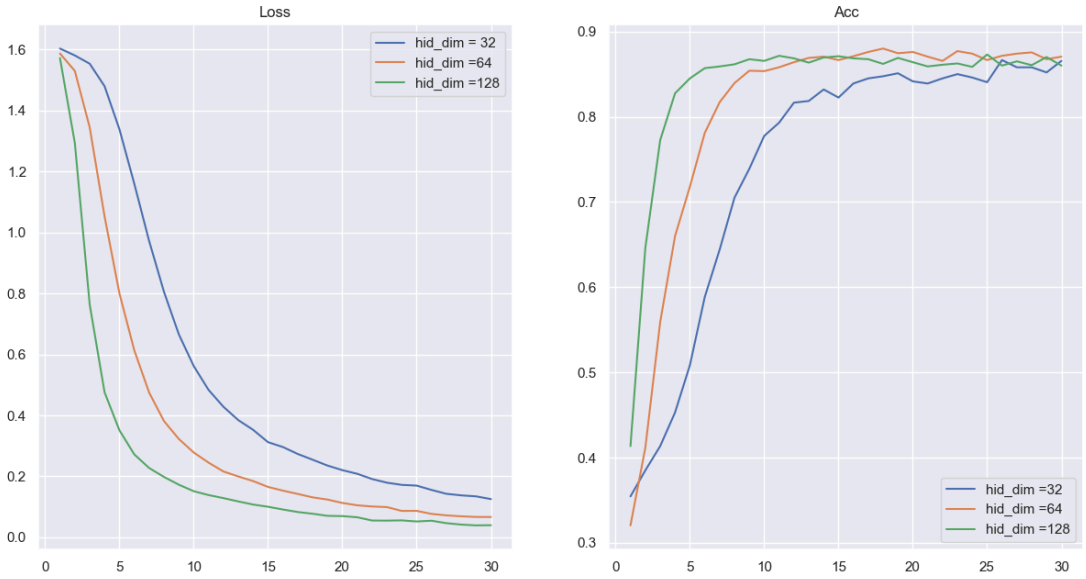

Figure 5: Testing Transformer with embedding dimensions of 32, 64 and 128, with a fixed hidden dimension of 64. Showing that embedding dimension = 128 performs the best training loss, while 64 has slightly best test accuracy

Aswell we can see that transformer embedding dimension has best training at embedding dimension 128 and best test accuracy at 64 dimension

3.1.2 Test of optimal RNN

While this serves as a good basic RNN architecture, its validation accuracy plateaus around 80%:

Figure 6: Training and validation loss as well validation accuracy of the RNN across 100 training epochs. Reaching a peak test accuracy at 84 %

While this is not bad for this simple architecture, it overfits quite a bit. To compensate for this, it needs to be regularized to improve generalization, one can do this in several ways, most commonly though either `dropout` or `masked_mean_pooling`.

Lets see how masked mean pooling affects the RNN's performance.

```python
def forward(self,
input_ids, lengths):
        emb = self.embedding(input_ids)

        packed = pack_padded_sequence(emb, lengths,
                                      batch_first=True,
                                      enforce_sorted=False)

                                      packed_out, _ = self.rnn(packed)
        output, _ = pad_packed_sequence(packed_out,

        batch_first=True)
        T_out = output.size(1)
        attention_mask = (input_ids[:, :T_out] != self.pad_idx)
        pooled = masked_mean_pool(output, attention_mask)

        pooled = self.dropout(pooled)
        return self.fc(pooled)
```

Code snippet 2: RNN + masked mean pooling
embedding → packing → rnn → padding → mask → mean → dropout → classify

While mean pooling is often assumed to improve generalization and reduce overfitting, the results show the opposite trend. Mean pooling decreased validation accuracy compared to using the final hidden state. This is fine, since the dair-ai/emotion mostly contains shorter sentences. However lets look at another issue with the pooling.

When testing with a RNN structure as shown above (Code snippet 1), one can compare using mean pooling on specific examples of text to see how something like mean pooling works on a RNN. Since mean pooling averages the embeddings out, it should in theory ruin the ordering of importance of tokens. That means that there should be little to no difference on sentences with reversed ordering and different meaning. While a standard RNN might recognize `i hate you` as `angry` and `you hate me` as `sad/fear`, A RNN using mean pooling, doesn't see any difference.

```
--- Testing TextRNN ---
  "i felt joy then i felt anger" → joy
  "i felt anger then i felt joy" → joy
  "i was happy but now i feel sad" → sadness
  "i was sad but now i feel happy" → joy
  "scared at first then surprised" → surprise
  "surprised at first then scared" → fear
  "today was full of fear and joy" → fear
  "today was full of joy and fear" → joy
```

Code snippet 3: RNN *without* mean pooling on sentences with switching meaning and ordering. It can be observed that the ordering of the words influence the output

Code snippet 3 shows the predictions of the TextRNN on sentence pairs where emotion order is reversed. The model generally captures the temporal structure, correctly leveraging the final hidden state h_n, which emphasizes later tokens in the sequence.

```
--- Testing TextRNN + Masked_Mean_Pooling ---
  "i felt joy then i felt anger" → sadness
  "i felt anger then i felt joy" → fear
  "i was happy but now i feel sad" → joy
  "i was sad but now i feel happy" → sadness
  "scared at first then surprised" → fear
  "surprised at first then scared" → surprise
  "today was full of fear and joy" → joy
  "today was full of joy and fear" → joy
```

Code snippet 4: RNN *with* mean pooling on sentences with switching meaning and ordering. Here it can be observed that the ordering of the words does not seem to impact the classification as much as without mean pooling

On Code snippet 4, the same experiment is repeated using an RNN with masked mean pooling. In contrast to the standard RNN, performance becomes less consistent on sentences with reversed ordering and different meaning. This suggests that averaging hidden states reduces sensitivity to the structure of a sequence, leading to weaker modeling of word order, and so weak emotional prediction.

The drawback of RNN seems to be that trying to generalize to improve accuracy counteracts the whole reason to use RNN in the first place (namely the sequence order). Moreover a big drawback of a simple (and standard) RNN architecture, is its sensitivity to long sequences. This is be an *vanishing/ exploding gradients*, which occur when the RNN works on longer sentences. Since the gradients are shared across layers, when there are many layer, the input becomes $x \cdot \nabla f^n$ which can quickly vanish or explode.

Now that the model has been fine-tuned to a great extend, lets test the model on the test set. Though is unseen data for the model, and should therefore give an unbiased estimate of performance. When testing in the test set the RNN model scored. This test is performed over 100 epochs of training.

$$\text{test accuracy} \approx 0.80 \rightarrow 80\%,$$

which roughly corresponds to the validation made earlier.

3.2 Model 2

The LSTM (Long-Short-Term-Memory) is a more specific type of RNN designed specifically to learn and remember information over longer sentences than a regular RNN. While regular RNN's does have longterm memory stored it weights, it perform poorly on longer sentences due to the *vanishing gradients* (as mentioned in Section 3.1). [1]

LSTM's work as good stepping between RNN's and more powerful modern architectures like transformers. They achieve greater recall by splitting the "knowledge" up in different areas, called *cell state* and *hidden state*. Where as the RNN architecture held both longterm and short term memory in the hidden state, the *cell state* and *hidden state* in the LSTM act as respectively long and short term memory, with the hidden states also acting as a gate to the cell state.

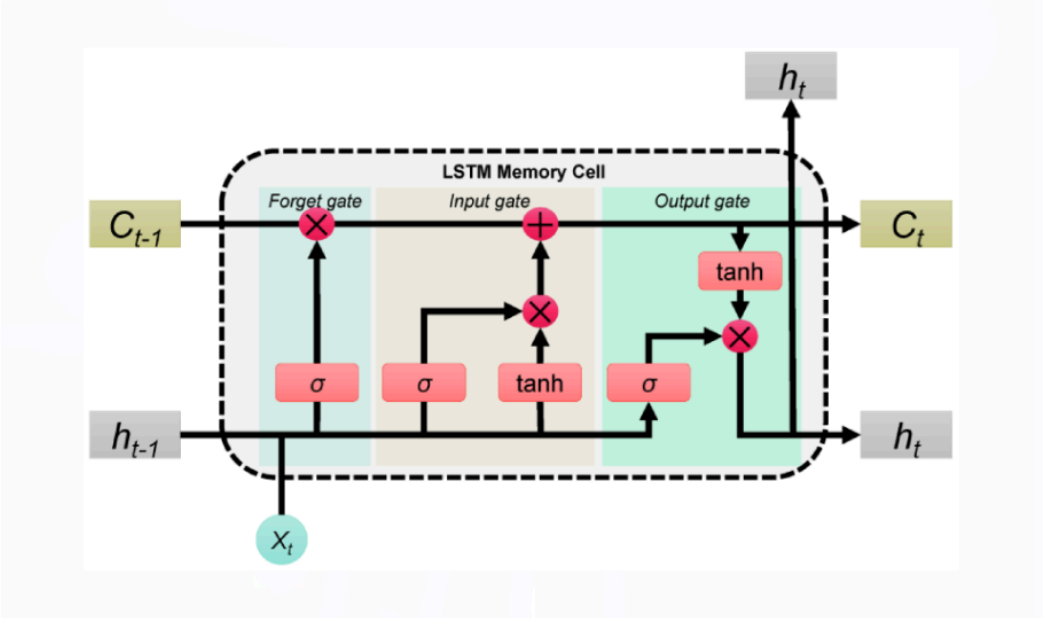

Figure 7: LSTM model architecture depicting the 3 stages of the LSTM model, the Forget gate, the Input gate and the Output gate [2]

The LSTM model has 3 gates hidden within its architecture. These three gates are exactly what makes the long-term-short-term memory work so well. The *forget gate* decides what information to discard from the cell state:

$$f_t = \sigma\big(W_f \cdot [h_{t-1}, x_t] + b_f\big)$$

The input gate decides what new information to write into the cell state:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\hat{C}_t = \tanh(W_C \cdot [h(t-1), x_t] + b_C)$$

The cell state is then updated by forgetting old information and adding new:

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \hat{C}_t$$

Finally, the output gate decides what part of the cell state to expose as the hidden state:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \cdot \tanh(C_t)$$

It's hard to measure exactly how much this increases the amount of context the model can capture, since it depends on a lot of other variables, but for this project, it can be tested against the RNN to see if it works better in practice.

The model architecture started with two layers, to keep it comparable to the RNN model, and thereby measure of much the difference of the modified internal architecture changed the performance.

3.2.1 Hidden and Embedding dimensions:

To find the optimal model, it was necessary to find the optimal amount of hidden and embedding layers, they were found experimentally. The following figure shows the performance of a LSTM with a fixed embedding dimension of 128, and hidden dimensions of 32, 64 and 128:

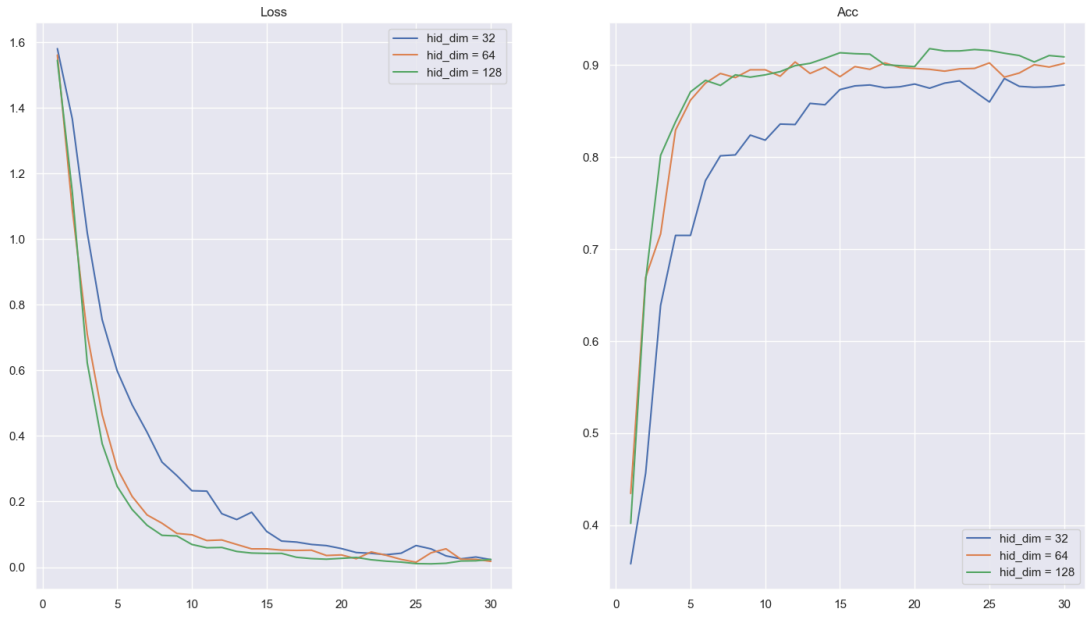

Figure 8: Testing LSTM with hidden dimensions of 32, 64 and 128, with a fixed embedding dimension of 128. Showing that hidden dimension = 128 performs the best.

From the testing it is clear that having a hidden dimension of 128 performed the best, but the increase in training was so high that that it was decided to keep it at 64, given the negligeable increase in performance.

The embedding dimensions could also be tested with a fixed hidden dimension of 64:

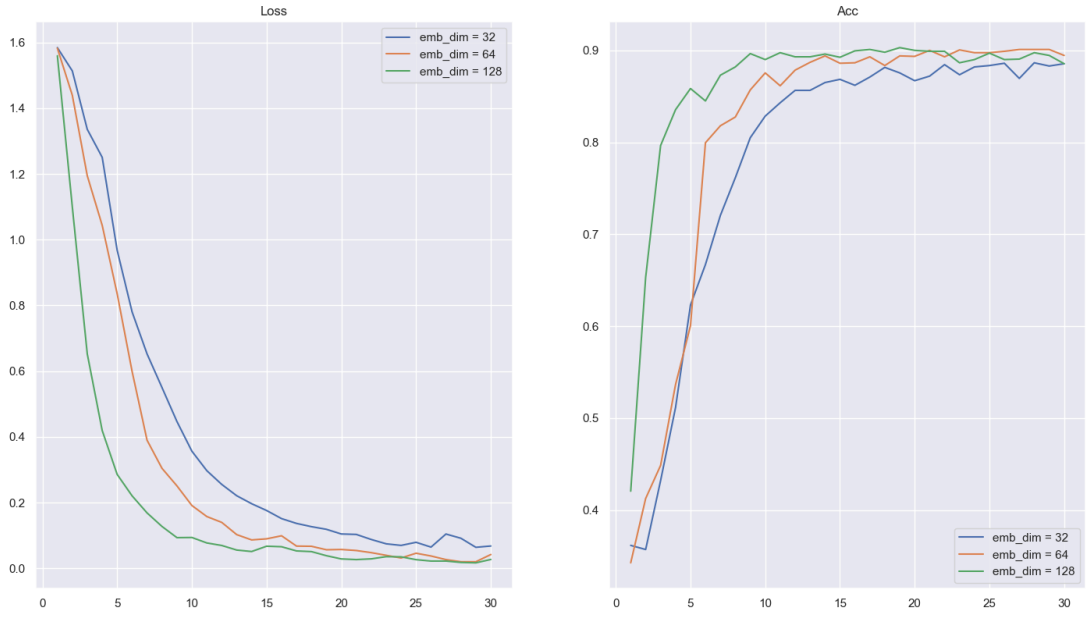

Figure 9: Testing LSTM with embedding dimensions of 32, 64 and 128, with a fixed hidden dimension of 64. Showing that embedding dimension = 128 performs the best.

From the testing it can be determined that an embedding of 128 performed the best, so this was chosen as the default parameter value.

3.2.2 Optimal LSTM Model

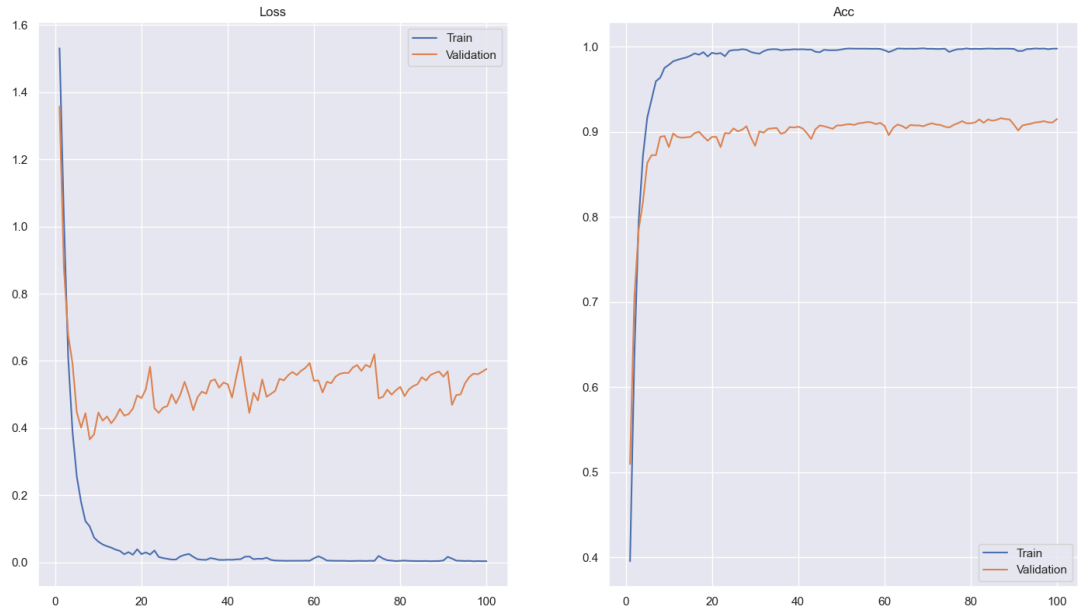

Figure 10: Training and validation loss as well validation accuracy of the LSTM across 100 training epochs. Despite overfitting in later epochs, the model reaches $\approx 91\%$ validation accuracy. This is quite better than the RNN.

The LSTM clearly outperforms the RNN, reaching around $\approx 92\%$ validation accuracy compared to roughly $\approx 80\%$ for the RNN. While the model does overfit with its training accuracy reaching 99-100% while validation plateaus at 92% it still generalizes significantly better than the RNN. The overfitting is likely due to the training set being relatively small compared to the model's capacity [3]. Now that the model has been fine-tuned lets also test this model on the test data.

Much like earlier in Section 3.2, the LSTM model has never seen the test data before the test, therefore is makes for a good unbiased estimation of performance. This test is performed over 100 epochs of training.

$$\text{test accuracy} \approx 0.906 \rightarrow \approx \mathbf{90.6}\%$$

This aligns well with the validation done earlier.
However, it became clear that two layers was not sufficient to implement linear probes later in the project, so it was changed to a total of 4 layers for later analysis.

3.3 Model 3

As shown RNN are powerful sequence models, but as mentioned above it can have drawbacks. One of these is sequential processing with parallelism, because the model depends on the hidden state before it, where it must process each token sequentially [4]. This makes it good at sequential data, but bad at utilizing hardware that can significantly decrease training time for other models, such as GPU's, as it computes many calculation carried out simultaneously, using parallelization. By these consideration a transformer architecture was implemented to test if it could out perform the current models. The transformer architecture is distinguished by its use of the attention mechanism, which relies on query, key, and value tensors to capture the contextual meaning and attention of each token in a sequence. Through this mechanism, tokens are mapped into an embedding space where each vector's position is influenced by every other token in the text.[3]

For each input, the transformer model learns three weight matrices and computes tree vectors q_k, v_k, k_k.

$$k_k = W_k \cdot x \qquad v_k = W_v \cdot x \qquad q_k = W_q \cdot x$$

Initially, getting information from tokens and finding which relations are strongest, captured in the attention weights, between key and query vector.

$$\hat{a}_i = q_k \cdot k_i$$

All the attention weight are summed and softmaxed To get a probability of each weight and normalize by each attention weight.

$$a_i = \frac{e^{a_i}}{\sum e^{\hat{a}_j}}$$

At last the transformer calculates the a vector z which is the weighted sum of all value vectors, where each value is weighted by how much correlation a token has to another.

$$z = \sum_i \hat{a}_i v_i$$

This single attention operation is *attention head*, which is a learned function that captures one type of relationship within the sequence. Since a single head is limited to attending to one relationship at a time. To attend to different types of relationships within a sequence, the attention operation is performed K times in parallel, each with its own learned

$$W_Q, W_K, W_V$$

matrices, where each head computes its own query, key and value vectors for each token. The output of each head is calculated:

$$Z = \text{softmax}\left(Q_k \frac{K_k^T}{\sqrt{d_k}}\right) V_k$$

Then outputs of all K heads are then concatenated into a vector and linearly transformed along the feature dimension.

3.3.1 Architecture

The architecture uses a simple layer norm normalization, normalized over the feature dimension by its mean and variance.

$$y = \frac{x - \mathbb{E}[x]}{\sqrt{\text{VAR}[X] + \varepsilon}} \cdot \gamma + \beta$$

while β, γ are learnable parameters applied for each input token on the attention vector. Layer norm creates a mean of 0 and variance of 1 for the vector. Afterwards the residual connection is applied. This helps fixing the *vanishing gradient* problem, as each token is applied in our module, which helps learnability. Applying normalization before residual connection is called *pre-norm*.

```python
class TransformerBlock(nn.Module):
    def __init__(self, d_model, d_key, n_heads, mlp_factor=4):
        super().__init__()


        # We need to init two layer norms because they  have parameters
        self.ln1 = nn.LayerNorm(d_model)
        self.attn = MultiHeadSelfAttention(d_model, d_key, n_heads)
        self.ln2 = nn.LayerNorm(d_model)

        # a feedforward module with one internal hidden layer
        self.mlp = nn.Sequential(
            nn.Linear(d_model, mlp_factor * d_model),
            nn.SiLU(),  # Swish activation function, f(x) = x * sigmoid(x)
            nn.Linear(mlp_factor * d_model, d_model)
        )

    def forward(self, x):
        # pre-norm
        x = self.attn(self.ln1(x)) + x
        x = self.mlp(self.ln2(x)) + x

        # post-norm
        # x = self.ln1(self.attn() + x)
        # x = self.ln2(self.mlp() + x)
        return x
```

At last, a feed-forward layer then processes each token independently, expanding to the model dimension, applying *SILU* for non-linearity, then compressing it back. This creates attention gathered into a greater token representation.

3.3.2 Optimal Transformer Model

Testing the transformer architecture gives a test accuracy at 88 %. This was done by using a pre-norm on the sub layer with layer norm. Even though through the literature, pre-norm is the most common, it would be interesting to see if it had a effect on the performance of the model using post norm. Post norm is additional adding normalization after residual connection.

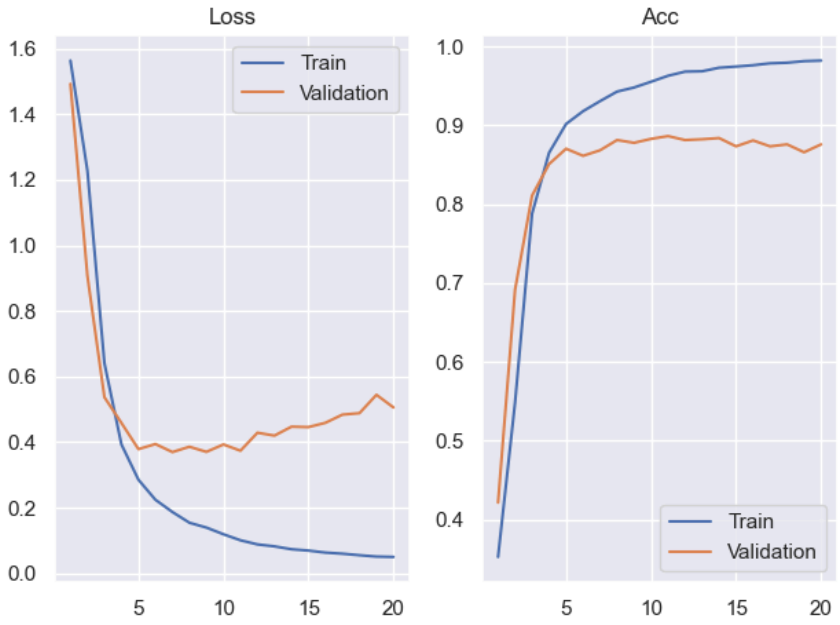

Figure 11: Train and validation loss, as well validation accuracy of the Transformer across 20 training epochs. Transformer model using self attention with pre-norm reaching peak at 88%.

The initial assumption was changing the normalization method on the attention layer wouldn't necessarily increase performance. But much like pre-norm stability, post norm also had a slight increase in performance, reaching 89% in peak test accuracy.

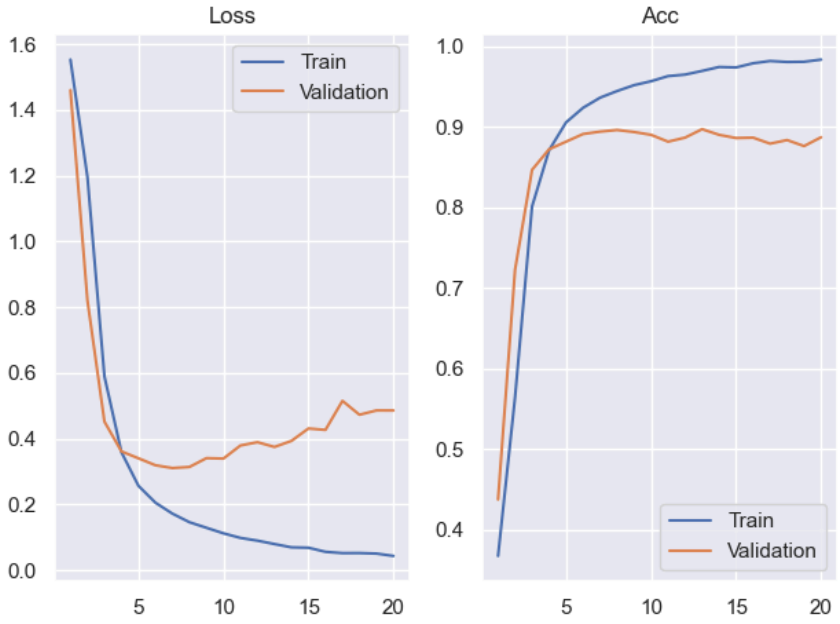

Figure 12: Train and validation loss, as well as train and validation accuracy of the Transformer across 20 training epochs. Transformer model using self attention with post-norm reaching peak at 89%.

Now that the transformer model has been fine-tuned, lets test it on several runs with post-norm:

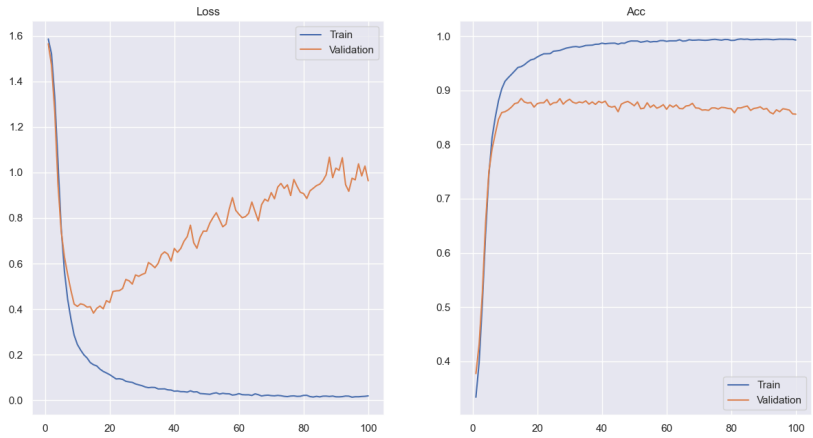

Figure 13: Training and validation loss as well validation accuracy of the Transformer across 100 training epochs. Increasing regularization had a slight decrease in performance

The final test is performed over 100 epochs. The following result shows

$$\text{test accuracy} \approx 0.88.1 \rightarrow \mathbf{88.1}\%$$

This accuracy almost corresponds to the validations made earlier, but with a slight decrease.

4 Task 4 - Analysis

Achieving high accuracy alone is not sufficient. Even though a model might perform well on a test, it is a well-known phenomenon in machine learning, where models may learn to associate certain inputs with labels without understanding their meaning. To examine this, this section tries to apply analysis techniques to inspect the internal representations of the model and get an understanding of when and why the models fail. Throughout this project, the LSTM architecture proved to be reliable and effective. Because of this the analysis will apply mostly to the LSTM-model.

4.1 Interpreter

Lets try and interpret the last hidden layer of the 2-layer LSTM. By training class-specific classifier, lets call this the **Interpreter**, on the last hidden layer, it is possible to test whether a concept like joy is actually encoded in the model's hidden states or not, and use this information to extract information of words and sentences.

Much like the final layer in the LSTM which actually classifies, the interpreter is trained on the final hidden layer of the model, more specifically it is trained on the locked hidden states from the model (gradients not updating). For the following tests, the interpreter has been training with a binary cross-entropy loss to predict weather a sentence expresses joy or not. After training, the interpreter vector `interpreter.fc.weight` should then point in the direction classified as joy:

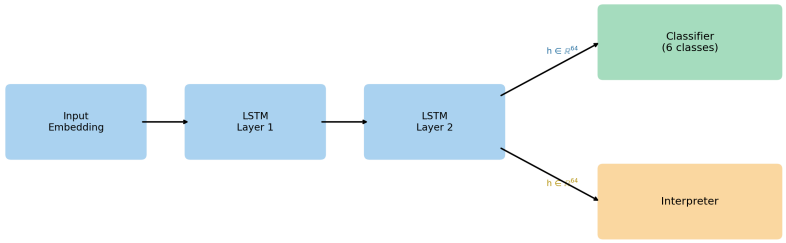

Figure 14: Simplified representation of the Interpreter replacing the classifier

The interpreter holds the weight vector w such that one can map the last hidden layer to a score which indicates to which extend it relates to joy. One way to interpret what the model has learned is to score every word in the vocabulary against the joy direction vector w. This is done by passing each word individually through the frozen LSTM and computing:

$$\text{score}(x_i) = \boldsymbol{w}^\top \ \text{LSTM}(x_i)$$

The resulting scores can then be sorted to reveal which words the model associates most strongly with joy and which it associates least:

```
1 top joy:                                                                output
  [('casual', 6.730369567871094), ('pleasant', 6.376668453216553),
  ('intelligent', 6.351129055023193), ('resolved', 6.247354507446289),
  ('superior', 6.246821403503418), ('carefree', 6.129107475280762), ('ecstatic',
  6.048050403594971), ('friendly', 6.04501485824585), ('delicious',
2 6.034866809844971), ('rich', 5.976566791534424), ('bouncy', 5.828309059143066),
  ('cute', 5.802940368652344), ('ansi', 5.686117172241211), ('divine',
  5.675238609313965), ('sincere', 5.671563148498535), ('entertained',
  5.642105579376221), ('innocent', 5.6107072830200195), ('mellow',
  5.5883588790893555), ('keen', 5.576539993286133), ('valuable',
  5.560917854309082)]
```

Code snippet 6: The 20 words most aligned with the joy direction

The top words (shown on Code snippet 6), such as `pleasant`, `carefree`, `ecstatic` and `friendly`, are intuitively associated with joy and positive emotion. Words like `ansi` however also appear, which is clearly noise. This likely scored highly, because it happened to occur alongside joy-labeled sentences in the training data and not because they actually carry emotional meaning.

```
1 bottom joy (-joy):                                                      output
  [('blocking', -5.167213439941406), ('uncertain', -5.106706619262695),
  ('shaken', -5.091213226318359), ('injury', -4.986476898193359), ('defeated',
  -4.956317901611328), ('timid', -4.9062275886535645), ('pressured',
  -4.7648091316223145), ('skeptical', -4.7271728515625), ('shaky',
  -4.611038684844971), ('doha', -4.572329044342041), ('wins',
2 -4.457796096801758), ('apprehensive', -4.443840026855469), ('frantic',
  -4.380986213684082), ('shy', -4.355367660522461), ('troubled',
  -4.3421525955200195), ('gere', -4.324879169464111), ('paranoid',
  -4.297036647796631), ('highschool', -4.295490741729736), ('abused',
  -4.28773832321167), ('needy', -4.250172138214111)]
```

Code snippet 7: The 20 words most opposed to the joy direction

Code snippet 7 shows the 20 most anti-joy pulling words. The bottom words paint an equally coherent picture, `defeated`, `pressured`, `frantic` and `troubled` are all associated with negative emotional meaning. Again, noise is present: `doha`, `gere` and `highschool` appear due to spurious correlations in the training data rather than any emotional meaning. It is important to remember that negative joy *does not* correspond to sadness, but rather implies that it is not associated with joy. A word could exist that only appear with sentences that are surprised, but does not itself imply surprise. This would have a low joy score, since it never is associated with a joy sentence.

Rather than only see a sentence at the end, one can also observe how the model's joy score evolves as it reads each token. By projecting the hidden state h_t onto w at each timestep, one gets a trajectory of how the model's "belief" in joy changes word by word:

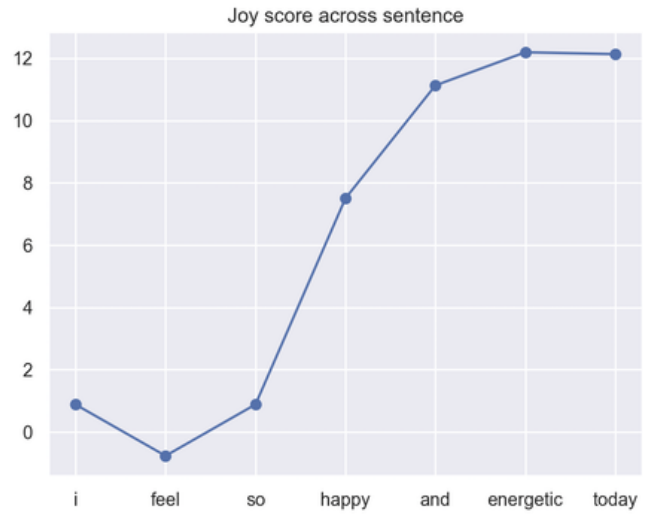

Figure 7:

Figure 15: Trajectory of the score over a sentence `"I feel so happy and energetic today"`. The score notably rises as joyful words occur.

This (Figure 15) is particularly interesting because it reveals when during the sentence the model effectively commits to a classification. With this test, its obvious that the decision to classify this sentence as joy, happens at the introduction of the words *happy*.

On the other hand if the sentence was to change sentiment over time it should decrease its score. This give a reason to investigate this some more. What happen when testing a sentence like `"I felt so happy and energetig today but now i feel doomed and troubled"`?

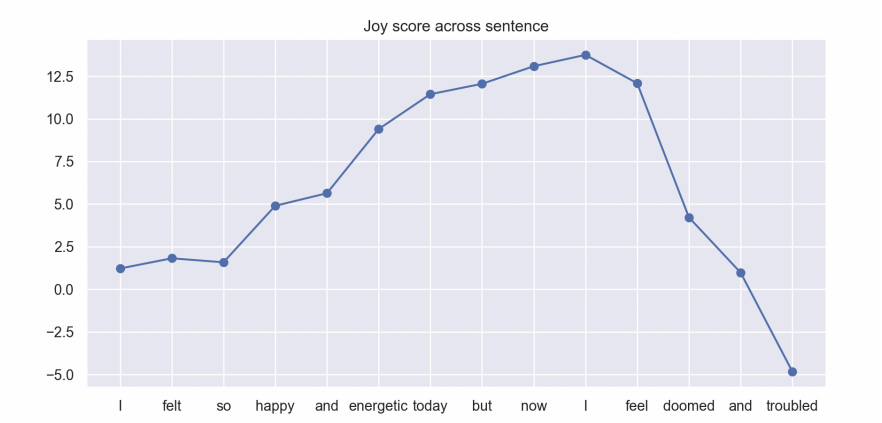

Figure 16: Trajectory of the score over a sentence `"I felt so happy and energetig today but now i feel doomed and troubled"`. One can see the increase but adding more negative associated word does show a significant decrease.

4.2 Linear Probing

So far, the extend of the analysis has been trying to represent/interpret a sentence at the last hidden state. To take this one step further and to understand the inner representations of the model better, techniques in mechanistic interpretability such as linear probing was used to try to yield information regarding these internal states. It is very hard to extract information from a sub-optimal model, so the following work has been done on the LSTM model, since it has a higher accuracy and therefore has a higher chance of encoding the knowledge.

4.3 Temporal probing

Using a 4 layer LSTM, it was first tried to train a binary probe for each token for each emotion. This was quite ambitious, but could work in theory. By first training a LSTM model, and then training a probe on each hidden outputs for a layer, the probe would then learn by comparing its prediction of the words semantic meaning from the hidden output, with the target of the sentence. By having enough data, the idea was that the probe would learn the meaning of the words given enough training, being able to cut trough the noise using "brute force". In practice, a binary classification with a very biassed dataset, such as optimization for a specific emotion like surprise which constitutes roughly 3.6% of the dataset, the probe quickly collapsed to only guess "not surprise". This was tried to remedy using *pos_weight* in the criterion, to fix the imbalance. *Pos_weight* is used to fix this imbalance by associating a positive weight to the positive class term in the BCELoss formula:

$$\ell(y_i, x_i) = -(y_i \cdot \log(x_i) + (1 - y_i) \cdot \log(1 - x_i))$$
$$\ell(y_i, x_i) = -(\text{pos\_weight} \cdot y_i \cdot \log(x_i) + (1 - y_i) \cdot \log(1 - x_i))$$

Where *pos_weight* is defined as:

$$\text{pos\_weight} = \frac{\#\text{not surprise}}{\#\text{surprise}}$$

But the model still defaulted to guessing one class. Time could have been spent trying to get it to work, but it was decided to instead focus the resources on other parts of the project.

4.4 Linear probing for each layer

The next implementation focused on training a linear probe on the last hidden layer, on each layer of the four layer LSTM, for each emotion and layer. This could yield information regarding at what stage the model transitions from capturing meaning of the words, to meanings of the entire sentence. After implementing a probe for each layer, that trains on the last hidden output of each layer, the final validation accuracy of the probe can be visualized:

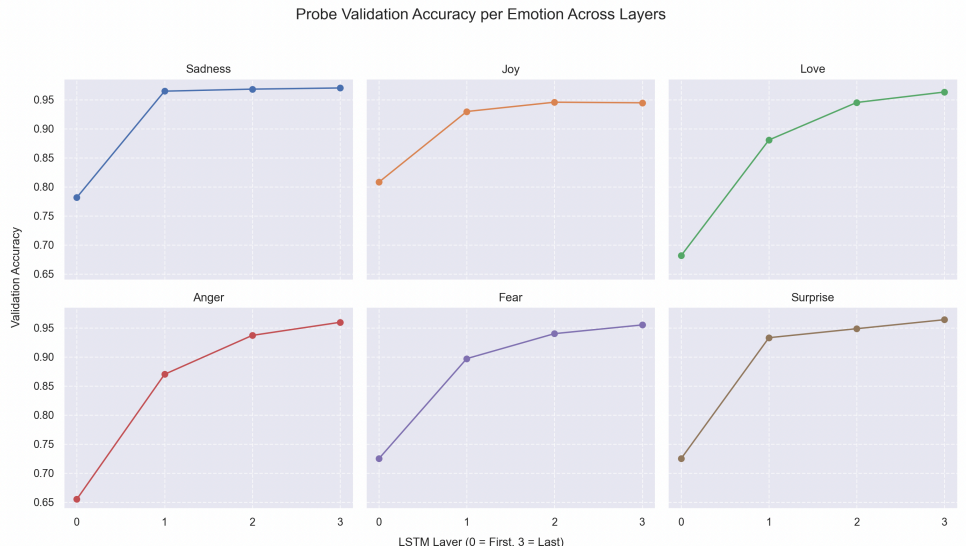

Figure 17: Probe validation accuracy for each emotion and layer in the LSTM model.

On Figure 17, one can see that it is possible for the probe to classify with decent accuracy most of the sentences for each of the layers.
The overall trend suggest that even in the earlier layers, specific knowledge regarding the emotion of the sentence can be extracted. This could be due to it not being a task that requires to much understanding from the model, before it can successfully classify the sentence. After having learned that keywords such as “casual”, “pleasant” and “carefree” generally imply that the sentence is positive, the model can achieve a relatively high score in classifying them without understanding more context in the sentence.

It is also observable that the 2 dominant classes, joy and sadness, are more quicker to generalize, then the less represented classes. This is likely to be due to the greater amount of training data for these classes, leading the model to faster find a clear indicator for these emotions. Even among these, sadness converges impressively fast, the probe being able to distinguishing sad / not sad with almost a 100% accuracy already after the second layer. This could indicate that the data, or traits within the dataset that allows the model evaluate them are not too complex. As will be demonstrated later, the models ability to catch concepts such as ‘not’ is severely lacking. If more complex models was used, and significant part of the dataset used these terms, it might be possible to measure the layers where the models start to comprehend them.

4.5 Understanding complex sentences

The models so far has mostly been performing well on the classification of less complex sentences. However in the case of more complex sentences, where meaning is not as linear as the sentence `"i am happy"`, it seems to perform worse. Take Figure 16 from Section 4.1 showed a sentence on the form `"i felt happy but..."`. To human brains this very much insinuates that something "not happy" will occur after the word but this is not picked up by the LSTM. Lets examine this a little bit further.

It is interesting to investigate wether the LSTM actually picks up on the meaning of words. Lets do a quick test first with the LSTM to establish a baseline:

```
1  "i feel happy" -> Joy score: 6.421                                    output
2  "i am not feel happy" -> Joy score: 6.606
```

Code snippet 8: Testing the word *not* and *so* to see if has an effect on sentence classification

It certainly seem like the LSTM model does not care about this, since it doesnt seem to understand the context in which words like `"not"`, `"but"` or even `"was"`. This would give a reason to examine the scores of the words some more. By simulating a forward pass of a sentence and then backpropagating, one can extract the gradients for each token. The gradients show how much a token is pushed towards a class. Because of this they can be seen as a "measure of importance" by taking the norm.

So the importance can be understood as:

$$s_i = \left\| \nabla_{\hat{y}_c} \right\| = \sqrt{\sum_{d \in D} \left(\frac{\partial \hat{y}_c}{\partial e_i} \right)^2}$$

Where e is the embedding of sentence x_i.

Now for each token in the sentence `"I am not happy today"` one can compute the gradient of the probe's output with respect to each token's embedding, which would indicating how much each word influences the classification.

```
[('not', 2.316), ('I', 2.184), ('happy', 2.064), ('today', 1.561), ('am', 1.425)]
```

Remarkably, *not* ranks highest, even above *happy*, implying that the LSTM has learned to attend to the negation rather than the sentiment-bearing word alone. This hints that the model's internal representations capture something beyond simple keyword matching. Note that this of course is not conclusive. The gradient norm reflects how sensitive the predicted class is to each token's representation, so whether the model has a symbolic understanding of negation is still hard to say. Nevertheless, it is a encouraging sign that the learned representations are not completely made up.

If the LSTM model actually capture the importance of words like `"not"`, there could be several reasons for the linear classification still fails like in shown in Code snippet 8. It might stem from the fact a linear mapping might not be expressive enough to capture the relationship between negation and sentiment. A natural next step would therefore be to replace the linear mapping with a small MLP, which might capture nonlinear structures such as negation.

```
1  i feel happy -> Joy score: 8.660
2  i do not feel happy -> Joy score: 9.114
```
output

Code snippet 9: Scoring by the interpreter, by using a MLP instead of a linear layer.

Unfortunately, replacing the linear map with a MLP, did not solve this. The reason for this remains unclear.

4.6 Word analysis

In Section 4.4 it was discussed that some emotions was easier to generalize. One reason might be emotionally charged words in the corpus are not equally weighted across classes. To investigate this, an interpreter is constructed for each emotion class, which is then used to score every word in the corpus. For each class, the top 20 highest-scoring words are identified and their average score is computed as a roughly measure of how strong and concentrated the emotional signal is:

```
1  sadness   : 1.0435
2  joy       : 1.8351
3  love      : 2.0546
4  anger     : 2.1059
5  fear      : 2.2048
6  surprise  : 2.4202
```
output

Code snippet 10: Averages of the top 20 scores of all classes

One could also examine the bottom 20:

```
1  sadness   : -6.7573
2  joy       : -6.9980
3  love      : -7.1310
4  anger     : -6.9609
5  fear      : -7.1297
6  surprise  : -7.0646
```
output

Code snippet 11: Averages of the bottom 20 scores of all classes

While there is notable difference in “weight” across the different classes (Code snippet 10 and Code snippet 11) it does not seem to serve as an explanation for variation in generalization. The weight imbalance, while interesting, does not seem to explain this case.

5 Discussion

5.1 Comparing models

Through the report, 3 models has been tested and compared for classification task of 6 labels. With these finding one can see that *LSTM* performs best with 100 epochs reaching a test accuracy peak around 92%, followed by the transformer with 89% and lastly RNN at 80%:

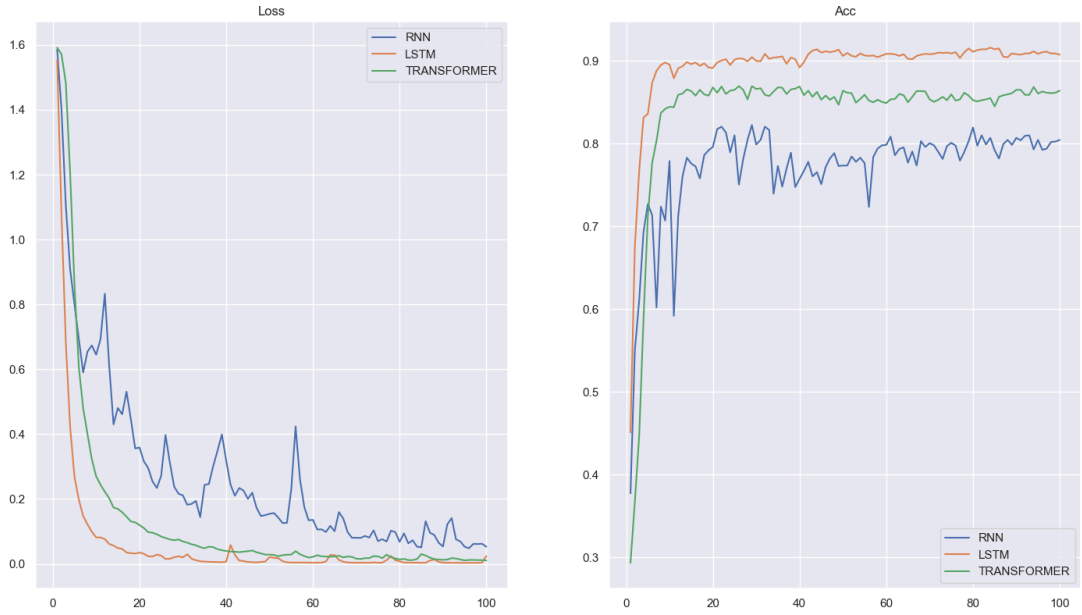

Figure 18: Comparing RNN, LSTM and Transformer with training loss and test accuracy.

Literature wise, one would expect the Transformer to win, since it is most commonly used architecture and self-attention has shown significantly greater results in different classification fields. One possible reason could be the small data size, as transformers usually need more training and validation data to learn properly, whereas sequence classifiers such as LSTM simply perform better on smaller data sizes. The following result shows the best test accuracy for all of the models:

Model	epochs	Peak Test Accuracy
Optimal RNN	100	**84%**
Optimal Transformer	100	**89%**
Optimal LSTM	100	**92%**

Table 1: Best encountered accuracies throughout testing.

5.2 Does RNN Struggle more with long sentences compared to LSTM

Theory seemed to suggest that LSTM models should be better at correctly classifying longer sentences. This seemed like interesting information, therefore code was made to gather the correct and incorrect classifications of the validation set from both the RNN and LSTM to test these assumptions. The number of correct and incorrect classified sentences were then sorted by token length, and then plotted for RNN and LSTM:

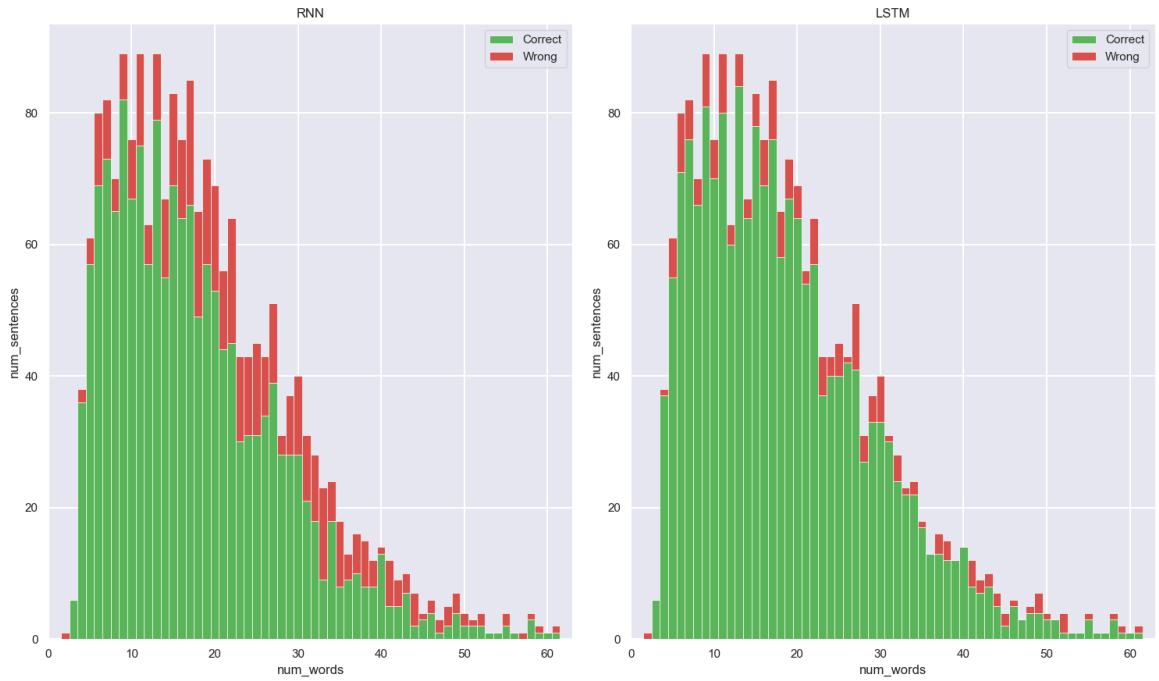

Figure 19: A comparison of how many correct/incorrect sentences for each amount of tokens the RNN and LSTM got correct. The most interesting part is the middle, between 20 and 50 words, where LSTM performs significantly better than the RNN.

It can be observed that at the start of the figures, the LSTM performs better. This is to be expected since the LSTM is an overall better model. For the ends of the model, 50 words and onwards, the performance is roughly the same, hinting that both models might struggle with especially long sentences. The most interesting part is the middle section, containing sentences with word lengths between 20-50, where the LSTM performs significantly better than the RNN. This might be due to the internal cell state in the LSTM, being able to remember key information from earlier in the sentence. Given more time, it could be interesting to test what factors effect the RNN's capacity to classify sentences of this length. It could be hypothesized that it might perform particularly bad on sentences of this length, where key information is located at the start. For example the sentence "I am happy today because [long context window] and that was all that happened" has the key word happy at the very start of the sentence, and this might impact the models ability to classify this sentence.

6 Conclusion

By these findings the report can conclude it was successfully to train 3 models for multi class classification by achieving a high test accuracy on LSTM by 92%. Additionally, the report has shown that it was successful with using mechanistic interpretability techniques to show that the models classifications are grounded in some internal structure rather than blindly believing that the model has found some magic statistical pattern in the data.

7 Who-did-What

The work allocation throughout this project was very fluid with all members working together on all parts of the project. That said, some members naturally took the lead of some parts.

- Johannes lead on the LSTM model
- Simon lead on the RNN
- Shuagib lead on the transformer model
- Analysis, dataloader and linear probing was a joint effort

Bibliography

[1] IBM, "What is long short-term memory (LSTM)?." [Online]. Available: https://www.ibm.com/think/topics/lstm

[2] dida, "What is an LSTM Neural Network?." 2026.

[3] A. C. Ian Goodfellow Yoshua Bengio, *Deep Learning*. 2015.

[4] Shawn, "RNN vs Transformer: A Deep Dive from Fundamentals to Applications." [Online]. Available: https://medium.com/@xiaxiami/rnn-vs-transformer-a-deep-dive-from-fundamentals-to-applications-ee4d700dd152